

CPP Module 08

You should evaluate 1 student in this team

Introduction

Please follow the rules below:

Remain polite, courteous, respectful, and constructive throughout the evaluation process. The community's well-being depends on it.

Work with the student or group being evaluated to identify potential issues in their project. Take time to discuss and debate the problems identified.

Understand that there may be differences in how peers interpret the project instructions and scope. Always keep an open mind and grade as honestly as possible. Pedagogy is effective only when peer evaluations are taken seriously.

Guidelines

Please follow the guidelines below:

Only grade the work submitted to the Git repository of the evaluated student or group.

Double-check that the Git repository belongs to the student(s) and that the project is the one expected. Ensure that git clone is used in an empty folder.

Carefully verify that no malicious aliases are used to deceive the evaluator into grading non-official content.

If applicable, review any scripts used for testing or automation together with the student.

If you haven't completed the assignment you're evaluating, read the entire subject before starting the evaluation.

Use the available flags to report an empty repository, a non-functioning program, a Norm error, or cheating. The evaluation process ends with a final grade of 0 (or -42 for cheating). However, except in cases of cheating, students are encouraged to review the work together to identify mistakes to avoid in the future.

Remember that no segfaults or other unexpected program terminations will be tolerated during the evaluation. If this occurs, the final grade is 0. Use the appropriate flag.

You should not need to edit any files except the configuration file, if it exists. If editing a file is necessary, explain the reasons to the evaluated student and ensure mutual agreement.

Verify the absence of memory leaks. All memory allocated on the heap must be properly freed before the program ends.

You may use tools like leaks, valgrind, or e_fence to check for memory leaks. If memory leaks are found, tick the appropriate flag.

Attachments

Please download the attachments below:

[subject.pdf](#)

Mandatory Part

Preliminary tests

If cheating is suspected, the evaluation stops here. Use the "Cheat" flag to report it. Take this decision calmly, wisely, and please, use this button with caution

Prerequisites

The code must compile with c++ and the flags -Wall -Wextra -Werror

Don't forget this project has to follow the C++98 standard. Thus,

C++11 (and later) functions or containers are NOT expected.

Any of these means you must not grade the exercise in question:

A function is implemented in a header file (except for template functions).

A Makefile compiles without the required flags and/or another compiler than c++.

Any of these means that you must flag the project with "Forbidden Function":

Use of a "C" function (*alloc, *printf, free).

Use of a function not allowed in the exercise guidelines.

Use of "using namespace <ns_name>" or the "friend" keyword.

Use of an external library, or features from versions other than C++98.

Yes

No

Improved addNumber function

Improved addNumber function

There's a way to add numbers that's more practical than calling the addNumber() function repeatedly.

Yes

No

Ex00: Easy find

As usual, there has to be the main function that contains enough tests to prove the program works as expected. If there isn't, do not grade this exercise. If any non-interface class is not in orthodox canonical class form, do not grade this exercise.

Template function

There is a templated function easyFind(T, int) that does what the subject requires.

It HAS to use STL algorithms.

If it does not (like manual search using iterators for example), count it as wrong.

Yes

No

Better tests

Better tests

There is a at least a main() function that has more tests than the ones
from the subject.

Yes

No

Ex02: Mutated abomination

As usual, there has to be the main function that contains enough tests to prove the program works as expected. If there isn't, do not grade this exercise.
If any non-interface class is not in orthodox canonical class form, do not grade this exercise.

MutantStack class

There is a MutantStack class that inherits from std::stack and offers
all of its member functions.

It has an iterator. Also, it is possible to do at least the operations
shown in the subject's examples using iterators.

Yes

No

Ex01: Span

As usual, there has to be the main function that contains enough tests to prove the program works as expected. If there isn't, do not grade this exercise.
If any non-interface class is not in orthodox canonical class form, do not grade this exercise.

Class and member functions

There is a class that complies with the constraints of the subject.

Its member functions use STL algorithms to find their result as much as possible. Finding the shortest span can't be done only by subtracting the two
lowest numbers (take a closer look at the subject example).

Yes

No

Bonus Part

no bonus

no bonus

no bonus

Yes

No

Ratings

OK

Outstanding

Empty Work

Incomplete Work

Invalid Compilation

Cheat

Crash

Concerning Situations

Leaks

Forbidden Functions